

Networking and Web Application step-by-step guide

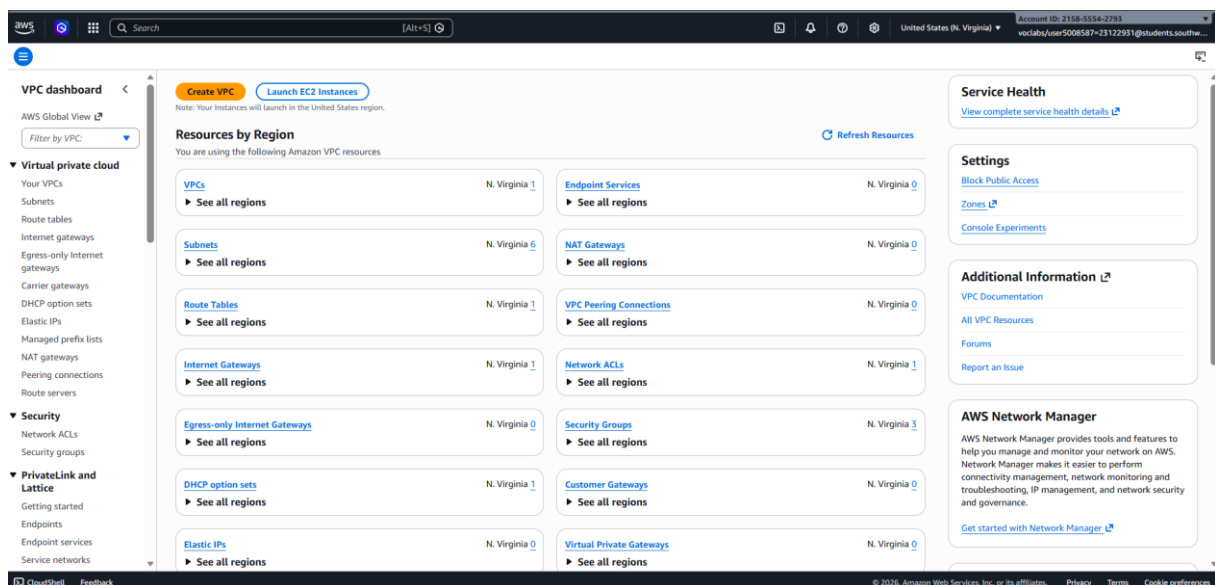
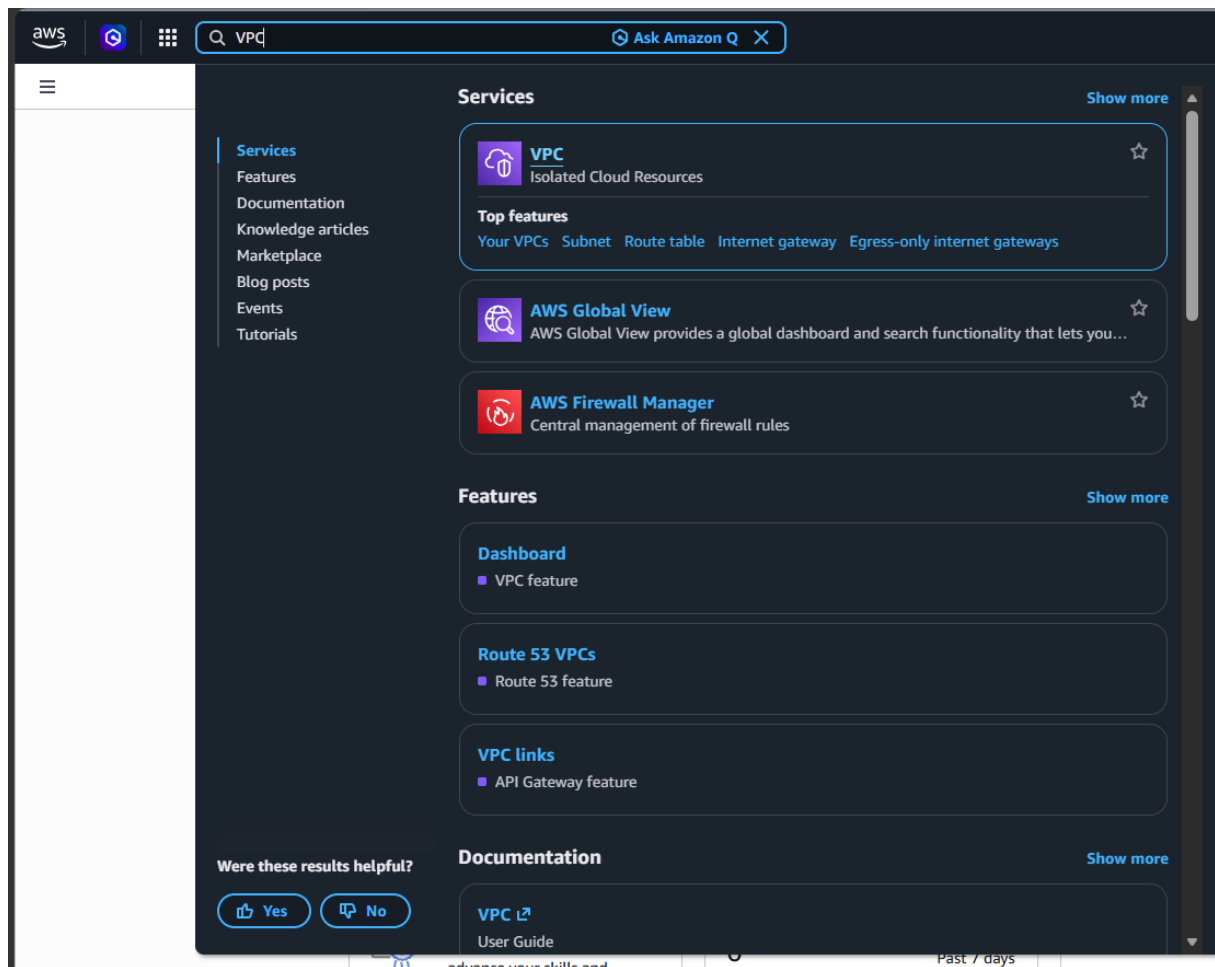
Introduction

This document describes how to set up the networking and web application for the EcoStream Energy cloud environment using Amazon Web Services. This guide shows how to create a custom Virtual Private Cloud named “23122931-ecostream-vpc” with its public and private subnets, route tables, and internet connectivity settings.

The document further discusses setting up an EC2 instance in the defined VPC environment and configuring it as a web application server accessible publicly via the Apache HTTP Server. Relevant screenshots, configuration settings, terminal commands, and validation outputs are provided to enable EcoStream staff to follow along and successfully reproduce the networking and web application deployment.

Step 1: The AWS search bar was used to search for “VPC”, and the VPC service under the Services category was selected to begin configuring the networking infrastructure required for the EcoStream environment.

The VPC Dashboard was opened successfully. From the dashboard, the “Create VPC” option was selected to begin creating a custom EcoStream Virtual Private Cloud containing the public and private subnets required to host the web application securely.



Step 2: The VPC configuration page was completed by selecting the “VPC and more” option, which automatically creates the networking resources required for the EcoStream environment. The VPC was named “23122931-ecostream-vpc” with an IPv4 CIDR block of “10.0.0.0/16”, while the default tenancy and no IPv6 option were retained.

The network was configured across “2 Availability Zones” to improve availability and fault tolerance. In addition, “2 public subnets” and “2 private subnets” were created to separate publicly accessible resources from internal private resources.

The NAT Gateway option was set to “None” to reduce unnecessary cost since the assessment brief only required a publicly accessible web application. DNS hostnames and DNS resolution were enabled to support proper communication and hostname resolution within the VPC environment.

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances. Mouse over a resource to highlight the related resources.

VPC settings [Resources to create](#) [Info](#)

Create only the VPC resource or the VPC and other networking resources.

☐ VPC only ☒ VPC and more

Name tag auto-generation [Info](#)

Enter a value for the Name tag. This value will be used to auto-generate Name tags for all resources in the VPC.

☒ Auto-generate

23122931-ecostream-vpc

IPv4 CIDR block [Info](#)

Choose the starting IP and the size of your VPC using CIDR notation.

10.0.0/16 65,536 IPs

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block

☐ Amazon-provided IPv6 CIDR block

Tenancy [Info](#)

Default

► **Encryption settings - optional**

Number of Availability Zones (AZs) [Info](#)

Choose the number of AZs in which to provision subnets. We recommend at least two AZs for high availability.

1 2 3

► **Customize AZs**

Number of public subnets [Info](#)

The number of public subnets to add to your VPC. Use public subnets for web applications that need to be publicly accessible over the internet.

0 2

Number of private subnets [Info](#)

The number of private subnets to add to your VPC. Use private subnets to secure backend resources that don't need public access.

0 2 4

► **Customize subnets CIDR blocks**

NAT gateways (\$) - updated [Info](#)

NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.

☒ None ☐ Regional - new ☐ Zonal

Introducing regional NAT gateway [Info](#)

AWS now offers a multi-AZ NAT Gateway, eliminating the need for separate NAT Gateways across availability zones.

VPC endpoints [Info](#)

Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

☐ None ☒ S3 Gateway

DNS options [Info](#)

☒ Enable DNS hostnames

☒ Enable DNS resolution

► **Additional tags**

Preview

VPC [Show details](#)

Your AWS virtual network

23122931-ecostream-vpc

Subnets (4)

Subnets within this VPC

us-east-1a

23122931-ecostream-vpc-subnet-

23122931-ecostream-vpc-subnet-

us-east-1b

23122931-ecostream-vpc-subnet-

23122931-ecostream-vpc-subnet-

Route tables (3)

Route network traffic to resources

23122931-ecostream-vpc-rtb-pa

23122931-ecostream-vpc-rtb-pr

23122931-ecostream-vpc-rtb-pr

Number of Availability Zones (AZs) [Info](#)

Choose the number of AZs in which to provision subnets. We recommend at least two AZs for high availability.

1 2 3

► **Customize AZs**

Number of public subnets [Info](#)

The number of public subnets to add to your VPC. Use public subnets for web applications that need to be publicly accessible over the internet.

0 2

Number of private subnets [Info](#)

The number of private subnets to add to your VPC. Use private subnets to secure backend resources that don't need public access.

0 2 4

► **Customize subnets CIDR blocks**

NAT gateways (\$) - updated [Info](#)

NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.

☒ None ☐ Regional - new ☐ Zonal

Introducing regional NAT gateway [Info](#)

AWS now offers a multi-AZ NAT Gateway, eliminating the need for separate NAT Gateways across availability zones.

VPC endpoints [Info](#)

Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

☐ None ☒ S3 Gateway

DNS options [Info](#)

☒ Enable DNS hostnames

☒ Enable DNS resolution

► **Additional tags**

Preview

VPC [Show details](#)

Your AWS virtual network

23122931-ecostream-vpc

Subnets (4)

Subnets within this VPC

us-east-1a

23122931-ecostream-vpc-subnet-

23122931-ecostream-vpc-subnet-

us-east-1b

23122931-ecostream-vpc-subnet-

23122931-ecostream-vpc-subnet-

Route tables (3)

Route network traffic to resources

23122931-ecostream-vpc-rtb-pa

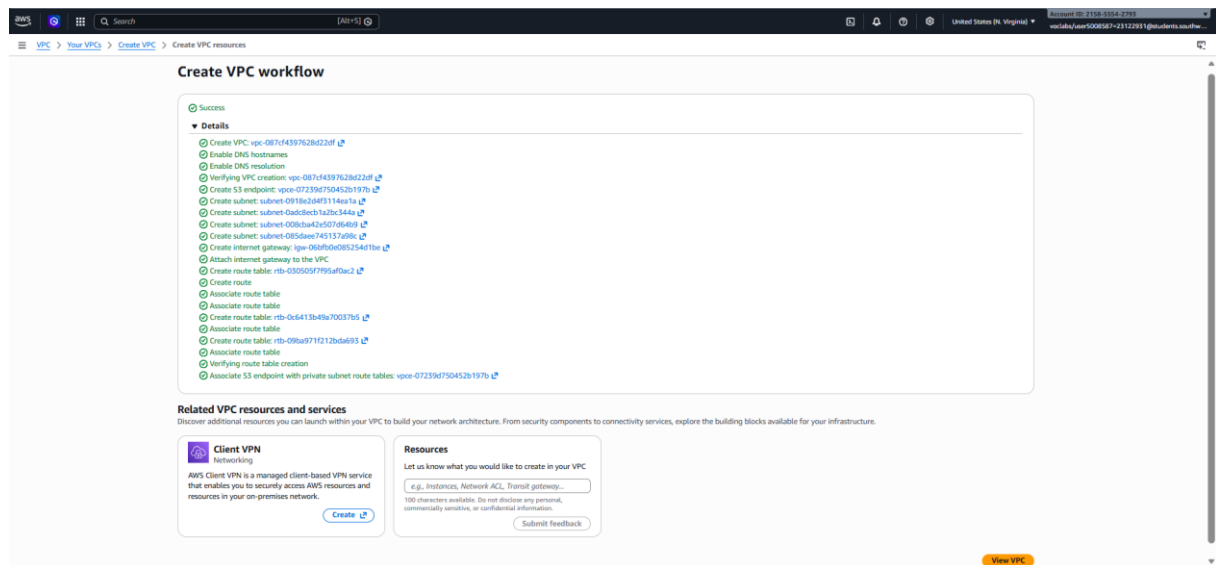
23122931-ecostream-vpc-rtb-pr

23122931-ecostream-vpc-rtb-pr

Step 3: The VPC deployment workflow page confirmed that the EcoStream Virtual Private Cloud and its associated networking resources were successfully created. AWS automatically provisioned the required components, including the VPC, public and private subnets, route tables, internet gateway, and DNS configurations based on the previously selected settings.

The workflow also verified that the route tables were correctly associated with the created subnets and that the internet gateway was successfully attached to provide public internet

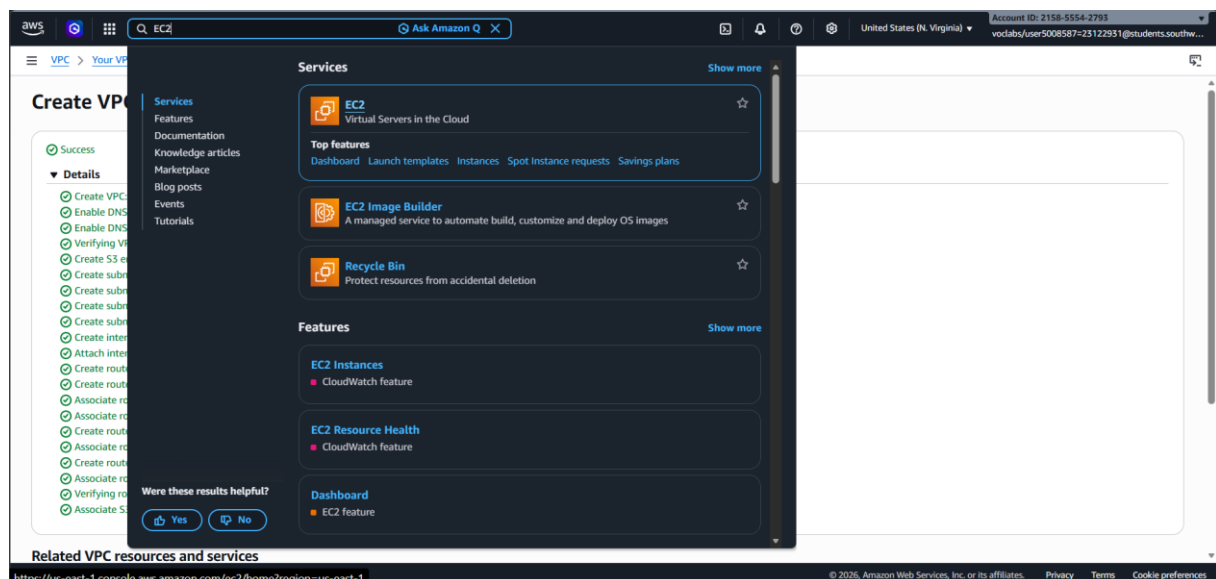
access for the web application instance. This step confirmed that the networking infrastructure required for hosting the EcoStream web application had been successfully configured and was ready for EC2 deployment.

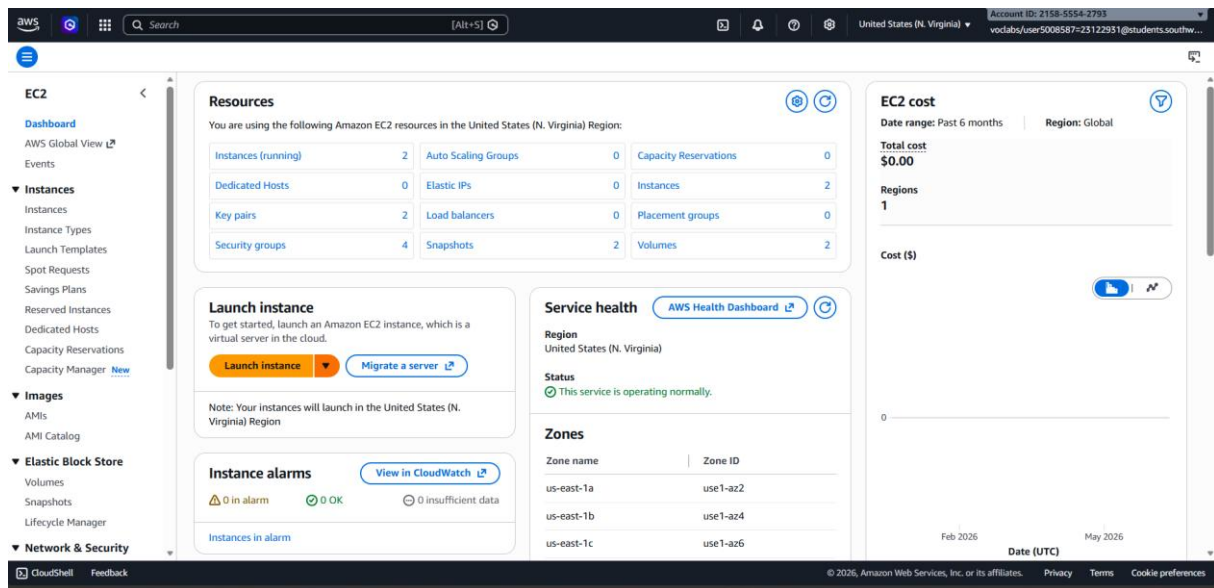


Step 4: After completing the VPC configuration, the AWS search bar was used to locate the EC2 service. The EC2 service was then selected to deploy the EcoStream web application server within the newly created VPC.

After navigating to the EC2 dashboard, the “Launch Instance” option was selected to create a new virtual server. The EC2 dashboard also confirmed that the AWS region in use was the “United States (N. Virginia)” and displayed the available EC2 resources and networking components associated with the account.

This step was necessary because the assessment required that the EcoStream web application be hosted on a publicly accessible EC2 instance within the newly created VPC infrastructure.





Step 5: The EC2 instance creation process was started by selecting the “Launch Instance” option from the EC2 dashboard. The instance was named “23122931-ecostream-web-instance” to clearly identify it as the EcoStream web application server, as required in the brief.

The “Amazon Linux 2023 AMI” was selected as the operating system because it provides a stable Linux environment suitable for hosting web applications. The selected instance type, “t3.nano”, was sufficient for the lightweight web application required for the assessment.

The previously created key pair “23122931-keypair” was selected to allow secure remote access to the EC2 instance. Under the network settings, the custom VPC “23122931-ecostream-vpc-vpc” was selected together with the public subnet “23122931-ecostream-vpc-subnet-public1-us-east-1a”. Auto-assign public IP was enabled so the web application could be accessed publicly through the internet.

A new security group was created with inbound rules allowing:

- HTTP traffic (Port 80) from anywhere (` 0.0.0.0/0`)
- SSH traffic (Port 22) from anywhere (` 0.0.0.0/0`)

These rules were necessary to allow public users to access the web application while also enabling remote administrative access to the server through the terminal.

Finally, the “Launch Instance” button was selected, and AWS confirmed that the EcoStream web application instance had been successfully launched.

Launch an instance

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags

Name

23122931-ecostream-web-instance

Add additional tags

Application and OS Images (Amazon Machine Image)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.

Search our full catalog including 1000s of application and OS images

RecentsMy AMIsQuick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
ami-0236922087fa98b6e (64-bit (x86), uefi-preferred) / ami-0576ef8c3440f536 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Summary

Number of instances1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.11...read more
ami-0236922087fa98b6e

Virtual server type (instance type)
t3.nano

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

Launch an instance

Instance type

t3.nano

Family: t3 2 vCPU 0.5 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0052 USD per Hour
On-Demand Windows base pricing: 0.0098 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0087 USD per Hour
On-Demand SUSE base pricing: 0.0052 USD per Hour

Additional costs apply for AMIs with pre-installed software

Key pair (login)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

23122931-keypair

Create new key pair

Network settings

VPC - required

vpc-087c4397628d22df (23122931-ecostream-vpc-vpc)
10.0.0.0/16

Subnet
subnet-0918e2d4f3114ea1a 23122931-ecostream-vpc-subnet-public1-us-east-1a
VPC: vpc-087c4397628d22df Owner: 219853542793 Availability Zone: us-east-1a (us-east-1-azc)
Zone type: Availability Zone IP addresses available: 4091 CIDR: 10.0.0.0/20

Auto-assign public IP
Enable

Firewall (security groups)
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Summary

Number of instances1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.11...read more
ami-0236922087fa98b6e

Virtual server type (instance type)
t3.nano

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

Launch an instance

Security group name - required

launch-wizard-3

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _ (underscores) (+&_[]@)

Description - required

launch-wizard-3 created 2026-05-18T21:35:02.569Z

Inbound Security Group Rules

Security group rule 1 (TCP: 80, 0.0.0.0/0)

Type

HTTP

Protocol

TCP

Port range

80

Source type

Anywhere

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Security group rule 2 (TCP: 22, 0.0.0.0/0)

Type

ssh

Protocol

TCP

Port range

22

Source type

Anywhere

Source

0.0.0.0/0

Description - optional

e.g. SSH for admin desktop

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Summary

Number of instances1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.11...read more
ami-0236922087fa98b6e

Virtual server type (instance type)
t3.nano

Firewall (security group)
New security group

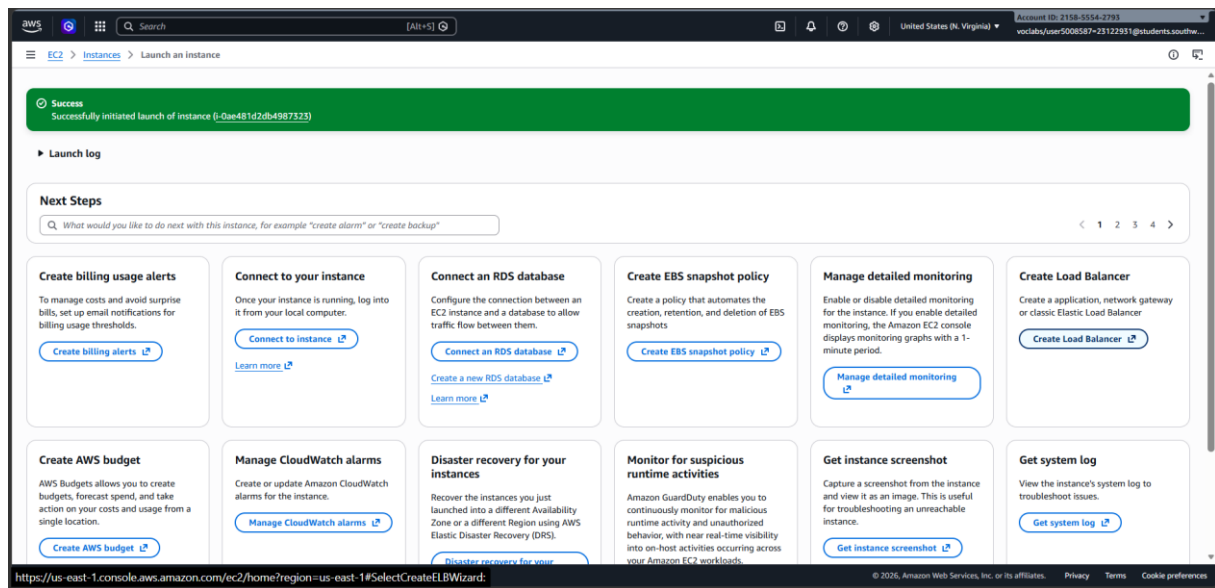
Storage (volumes)
1 volume(s) - 8 GiB

Cancel

Launch instance

Preview code

CloudShell Feedback © 2016, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

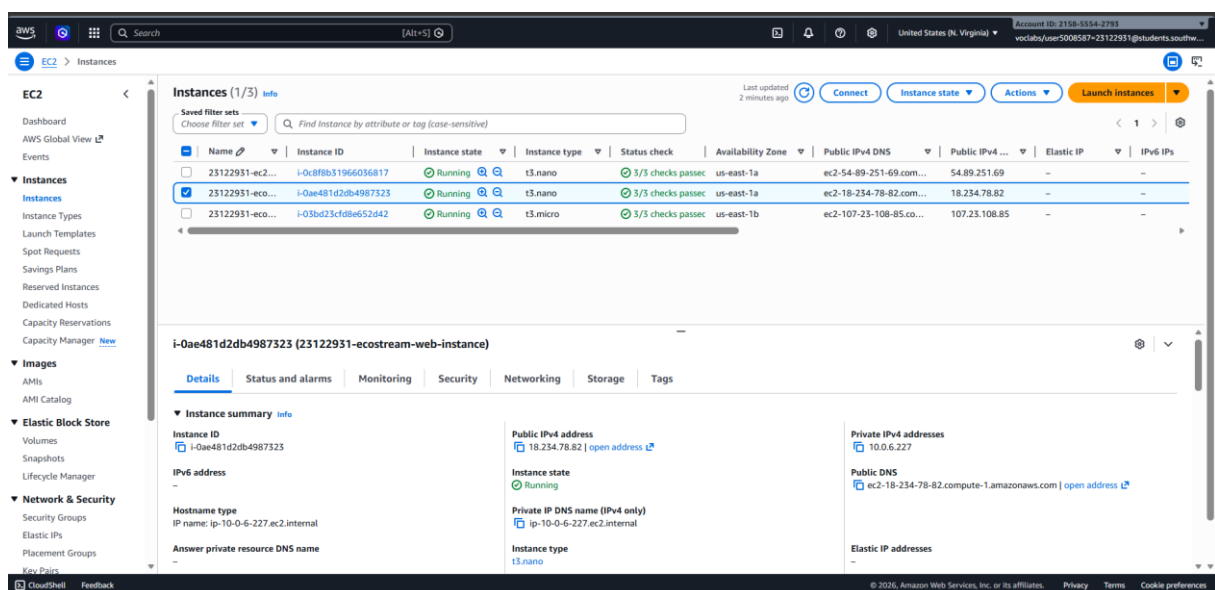


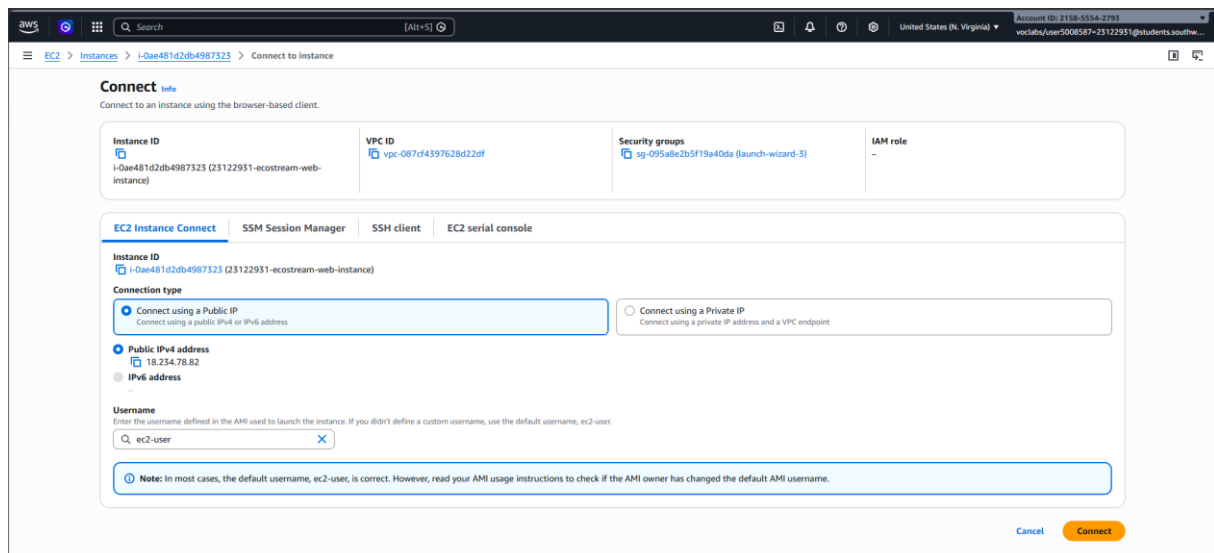
Step 6: After the EC2 instance was launched, the Instances page was opened to verify that the “23122931-ecostream-web-instance” was successfully running. The instance status checks showed “3/3 checks passed”, confirming that the server had been deployed correctly within the EcoStream VPC environment.

The instance details section also displayed the assigned public IPv4 address and private IP address, confirming that the instance was connected to both the internet and the internal VPC network.

The instance was then selected, and the “Connect” option was used to open the EC2 Instance Connect interface. The “EC2 Instance Connect” tab was selected with the default username “ec2-user” retained. The connection was configured using the instance’s public IP address to allow secure browser-based terminal access to the Linux server.

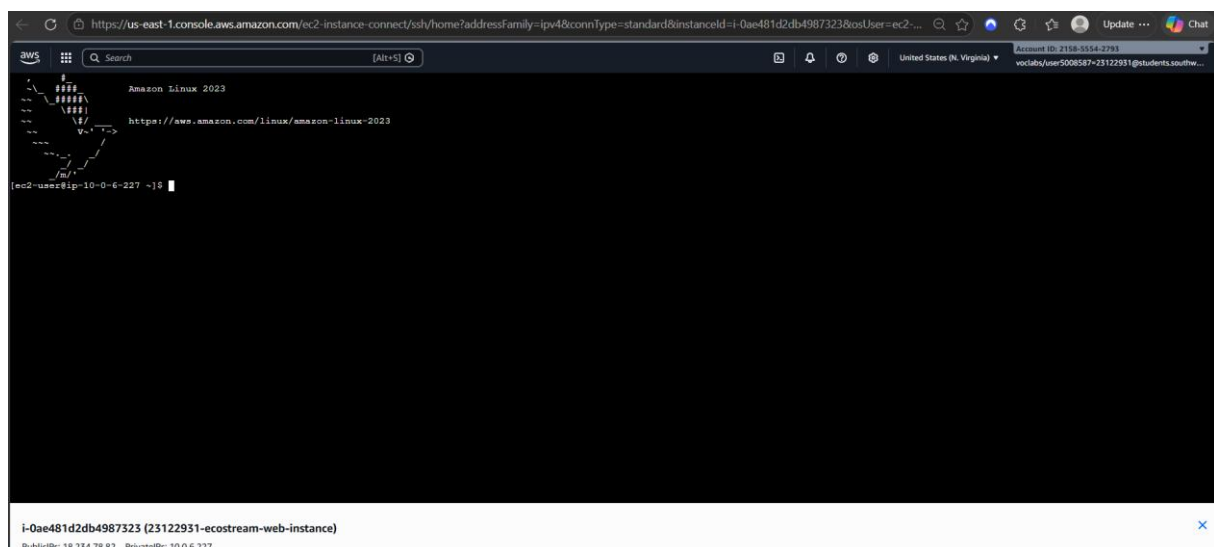
This step was necessary to access the EC2 terminal and install the Apache web server required to host the EcoStream web application.





Step 7: The EC2 Instance Connect terminal was successfully opened for the “23122931-ecostream-web-instance”. The browser-based Linux terminal provided secure remote access to the Amazon Linux 2023 server using the default “ec2-user” account.

This step confirmed that the EC2 instance was fully operational and ready for web server configuration. The terminal environment was then prepared for installing and deploying the EcoStream web application using Apache HTTP Server commands.



Step 8: The EC2 terminal session was elevated to root administrator access using the “sudo su –” command to allow system-level software installation and configuration. After this, the “yum update -y” command was executed to update the Amazon Linux package repositories and ensure the server environment was running the latest available system packages.

The successful completion message confirmed that the system update process executed correctly and that the server environment was ready for Apache web server installation and web application deployment.

```
https://us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-0ae481d2db4987323&osUser=ec2-...
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-0-6-227 ~]$ sudo su -
[root@ip-10-0-6-227 ~]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Last metadata expiration check: 0:00:01 ago on Mon May 18 22:10:15 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-6-227 ~]#
```

i-0ae481d2db4987323 (23122931-ecostream-web-instance)
PublicIP: 18.234.78.82 PrivateIP: 10.0.6.227

Step 9: The Apache HTTP Server installation was initiated with the command “yum install -y httpd”. This command installed the Apache web server package together with all required dependencies on the Amazon Linux EC2 instance.

The terminal output displayed the download, installation, and verification stages of the required packages, confirming that the Apache HTTP Server “httpd” and its supporting components were successfully installed on the server. The completion message at the end of the installation process confirmed that the web server environment had been successfully configured and was ready for web application deployment.

```
https://us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-0ae481d2db4987323&osUser=ec2-...
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-0-6-227 ~]$ sudo su -
[root@ip-10-0-6-227 ~]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Last metadata expiration check: 0:00:01 ago on Mon May 18 22:10:15 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-6-227 ~]# yum install -y httpd
Last metadata expiration check: 0:01:13 ago on Mon May 18 22:10:15 2026.
Dependencies resolved.

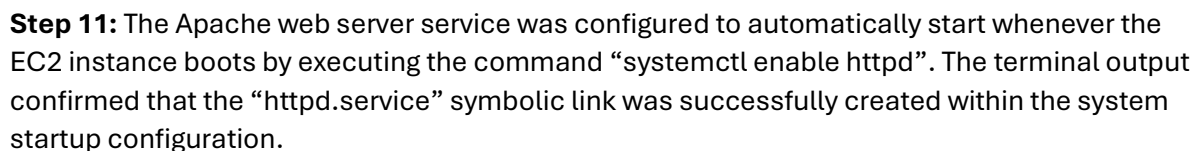
Package Architecture Version Repository Size
Installing:
httpd x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 47 k
Installing dependencies:
apr x86_64 1.7.5-1.amzn2023.0.4 amazonlinux 129 k
apr-util x86_64 1.6.3-1.amzn2023.0.2 amazonlinux 97 k
apr-util-ldap x86_64 1.6.3-1.amzn2023.0.2 amazonlinux 12 k
generic-logos-httpd noarch 18.0.0-12.amzn2023.0.3 amazonlinux 19 k
httpd-core x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 1.4 M
httpdfilesystem noarch 2.4.66-1.amzn2023.0.1 amazonlinux 13 k
httpd-tools x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 81 k
libbrotli x86_64 1.0.9-4.amzn2023.0.2 amazonlinux 315 k
mailcap noarch 2.1.49-3.amzn2023.0.3 amazonlinux 32 k
Installing weak dependencies:
apr-util-openssl x86_64 1.6.3-1.amzn2023.0.2 amazonlinux 15 k
mod_http2 x86_64 2.0.27-1.amzn2023.0.3 amazonlinux 166 k
mod_lua x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 60 k

Verifying : mod_lua-2.4.66-1.amzn2023.0.1.x86_64 13/13
Installing:
apr-1.7.5-1.amzn2023.0.4.x86_64 apr-util-1.6.3-1.amzn2023.0.2.x86_64 apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64 apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch httpd-2.4.66-1.amzn2023.0.1.x86_64 httpd-core-2.4.66-1.amzn2023.0.1.x86_64 httpdfilesystem-2.4.66-1.amzn2023.0.1.noarch
httpd-tools-2.4.66-1.amzn2023.0.1.x86_64 libbrotli-1.0.9-4.amzn2023.0.2.x86_64 mailcap-2.1.49-3.amzn2023.0.3.noarch mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64
Complete!
[root@ip-10-0-6-227 ~]#
```

i-0ae481d2db4987323 (23122931-ecostream-web-instance)
PublicIP: 18.234.78.82 PrivateIP: 10.0.6.227

Step 10: The command (`echo "Welcome 23122931! Welcome to EcoStream, the largest renewable energy company in the United Kingdom." > /var/www/html/index.html`) was executed

This step was important because the “index.html” file serves as the main webpage users see when accessing the EC2 instance via its public IP address. The successful execution of the command confirmed that the required web application content had been deployed to the Apache web server directory.



```
https://us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressfamily=ipv4&connType=standard&instanceid=i-0ae481d2db4987323&osUser=ec2...
aws [Alt+S] United States (N. Virginia) Account ID: 2185-3354-2793 vccatb/user5008587-23122931@students.usouth...

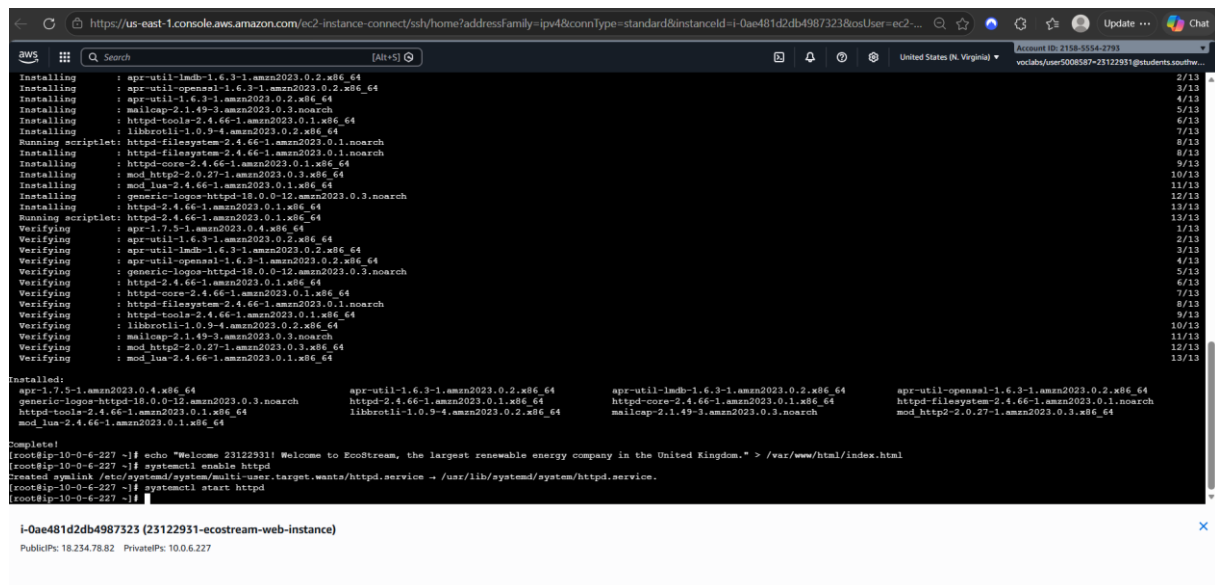
Installing : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Installing : apr-util-1.6.3-1.amzn2023.0.2.x86_64 2/13
Installing : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 3/13
Installing : apr-util-1.6.3-1.amzn2023.0.2.x86_64 4/13
Installing : mailcap-2.1.49-3.amzn2023.0.3.noarch 5/13
Installing : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64 6/13
Installing : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 7/13
Running scriptlet: httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch 8/13
Installing : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch 8/13
Installing : httpd-core-2.4.66-1.amzn2023.0.1.x86_64 9/13
Installing : mod_http2-2.0.27-1.amzn2023.0.3.x86_64 10/13
Installing : mod_lua-2.4.66-1.amzn2023.0.1.x86_64 11/13
Installing : qemucore-logs-httpd-18.0-12.amzn2023.0.3.noarch 12/13
Installing : httpd-2.4.66-1.amzn2023.0.1.x86_64 13/13
Running scriptlet: httpd-2.4.66-1.amzn2023.0.1.x86_64 13/13
Verifying : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Verifying : apr-util-1.6.3-1.amzn2023.0.2.x86_64 2/13
Verifying : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 3/13
Verifying : apr-util-1.6.3-1.amzn2023.0.2.x86_64 4/13
Verifying : qemucore-logs-httpd-18.0-12.amzn2023.0.3.noarch 5/13
Verifying : httpd-2.4.66-1.amzn2023.0.1.x86_64 6/13
Verifying : httpd-core-2.4.66-1.amzn2023.0.1.x86_64 7/13
Verifying : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch 8/13
Verifying : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64 9/13
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 10/13
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch 11/13
Verifying : mod_http2-2.0.27-1.amzn2023.0.3.x86_64 12/13
Verifying : mod_lua-2.4.66-1.amzn2023.0.1.x86_64 13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
qemucore-logs-httpd-18.0-12.amzn2023.0.3.noarch
httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Complete!
[root@ip-10-0-6-227 ~]# echo "Welcome 23122931! Welcome to EcoStream, the largest renewable energy company in the United Kingdom." > /var/www/html/index.html
[root@ip-10-0-6-227 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-6-227 ~]#
```

Step 12: The Apache web server was started using the command “systemctl start httpd”. This command started the Apache HTTP service on the EC2 instance, making the EcoStream web application accessible via the server’s public IP address.

The successful execution of the command confirmed that the “httpd” service started successfully and that the web server was ready to host the EcoStream web application.



```
Installing : apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
Installing : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
Installing : apr-util-1.6.3-1.amzn2023.0.2.x86_64
Installing : mailcap-2.1.49-3.amzn2023.0.3.noarch
Installing : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
Installing : libbrotli-1.0.9-4.amzn2023.0.2.x86_64
Running scriptlet: httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
Installing : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
Installing : httpd-core-2.4.66-1.amzn2023.0.1.x86_64
Installing : mod_http2-2.0.27-1.amzn2023.0.3.x86_64
Installing : mod_lua-2.4.66-1.amzn2023.0.1.x86_64
Installing : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
Running scriptlet: httpd-2.4.66-1.amzn2023.0.1.x86_64
Verifying : apr-1.7.5-1.amzn2023.0.4.x86_64
Verifying : apr-util-1.6.3-1.amzn2023.0.2.x86_64
Verifying : apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
Verifying : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
Verifying : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
Verifying : httpd-2.4.66-1.amzn2023.0.1.x86_64
Verifying : httpd-core-2.4.66-1.amzn2023.0.1.x86_64
Verifying : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
Verifying : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch
Verifying : mod_http2-2.0.27-1.amzn2023.0.3.x86_64
Verifying : mod_lua-2.4.66-1.amzn2023.0.1.x86_64

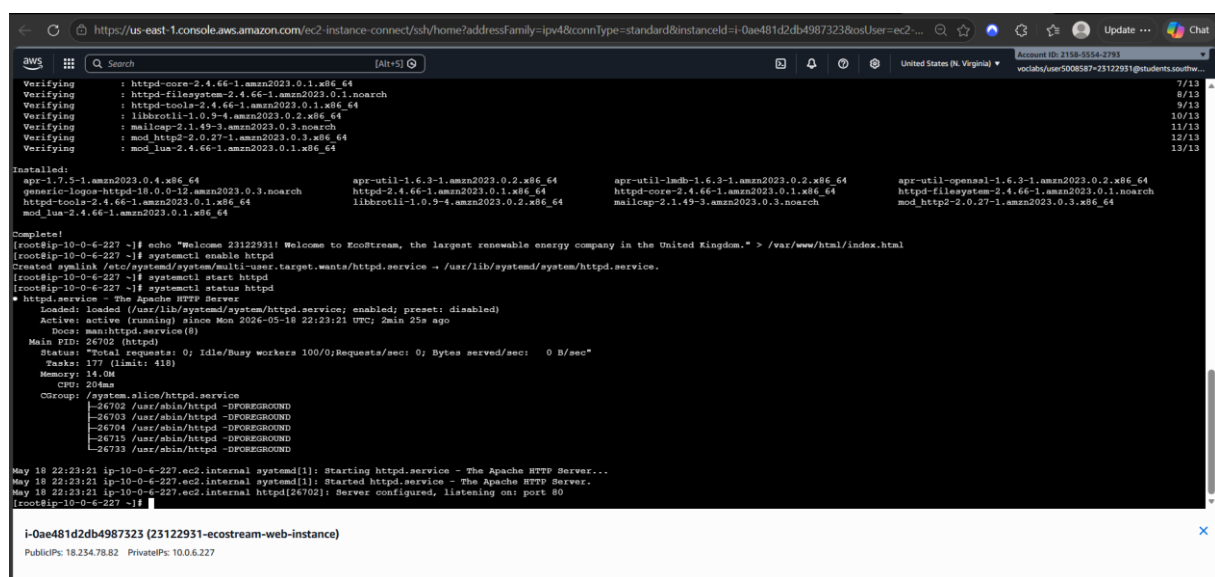
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64          apr-util-1.6.3-1.amzn2023.0.2.x86_64    apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64    apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch    httpd-2.4.66-1.amzn2023.0.1.x86_64    httpd-core-2.4.66-1.amzn2023.0.1.x86_64    httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
httpd-tools-2.4.66-1.amzn2023.0.1.x86_64          libbrotli-1.0.9-4.amzn2023.0.2.x86_64    mailcap-2.1.49-3.amzn2023.0.3.noarch    mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Complete!
[root@ip-10-0-6-227 ~]# echo "Welcome 23122931! Welcome to EcoStream, the largest renewable energy company in the United Kingdom." > /var/www/html/index.html
[root@ip-10-0-6-227 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-6-227 ~]# systemctl start httpd
[root@ip-10-0-6-227 ~]#
```

i-Oae481d2db4987323 (23122931-ecostream-web-instance)
PublicIPs: 18.234.78.82 PrivateIPs: 10.0.6.227

Step 13: The status of the Apache web server was verified using the command “systemctl status httpd”. The terminal output confirmed that the “httpd.service” was successfully loaded and running on the EC2 instance.

The status message displayed “active (running)”, confirming that the Apache HTTP Server was operating correctly and listening on Port 80 for incoming web traffic. The output also confirmed that the EcoStream web application server was fully operational and ready to serve web content to the public via the instance’s public IP address.



```
Verifying : httpd-core-2.4.66-1.amzn2023.0.1.x86_64
Verifying : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
Verifying : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch
Verifying : mod_http2-2.0.27-1.amzn2023.0.3.x86_64
Verifying : mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64          apr-util-1.6.3-1.amzn2023.0.2.x86_64    apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64    apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch    httpd-2.4.66-1.amzn2023.0.1.x86_64    httpd-core-2.4.66-1.amzn2023.0.1.x86_64    httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
httpd-tools-2.4.66-1.amzn2023.0.1.x86_64          libbrotli-1.0.9-4.amzn2023.0.2.x86_64    mailcap-2.1.49-3.amzn2023.0.3.noarch    mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Complete!
[root@ip-10-0-6-227 ~]# echo "Welcome 23122931! Welcome to EcoStream, the largest renewable energy company in the United Kingdom." > /var/www/html/index.html
[root@ip-10-0-6-227 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-6-227 ~]# systemctl start httpd
[root@ip-10-0-6-227 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-05-18 22:23:21 UTC; 2min 25s ago
     Docs: man:htttpd.service(8)
    Main PID: 26702 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec 0; Bytes served/sec 0 B/sec"
      Tasks: 177 (limit: 418)
     Memory: 14.0M
        CPU: 204ms
    CGroup: /system.slice/httpd.service
            └─26702 /usr/sbin/httpd -DFOREGROUND
              └─26703 /usr/sbin/httpd -DFOREGROUND
                └─26704 /usr/sbin/httpd -DFOREGROUND
                  └─26713 /usr/sbin/httpd -DFOREGROUND
                    └─26732 /usr/sbin/httpd -DFOREGROUND

May 18 22:23:21 ip-10-0-6-227.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
May 18 22:23:21 ip-10-0-6-227.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
May 18 22:23:21 ip-10-0-6-227.ec2.internal httpd[26702]: Server configured, listening on: port 80
[root@ip-10-0-6-227 ~]#
```

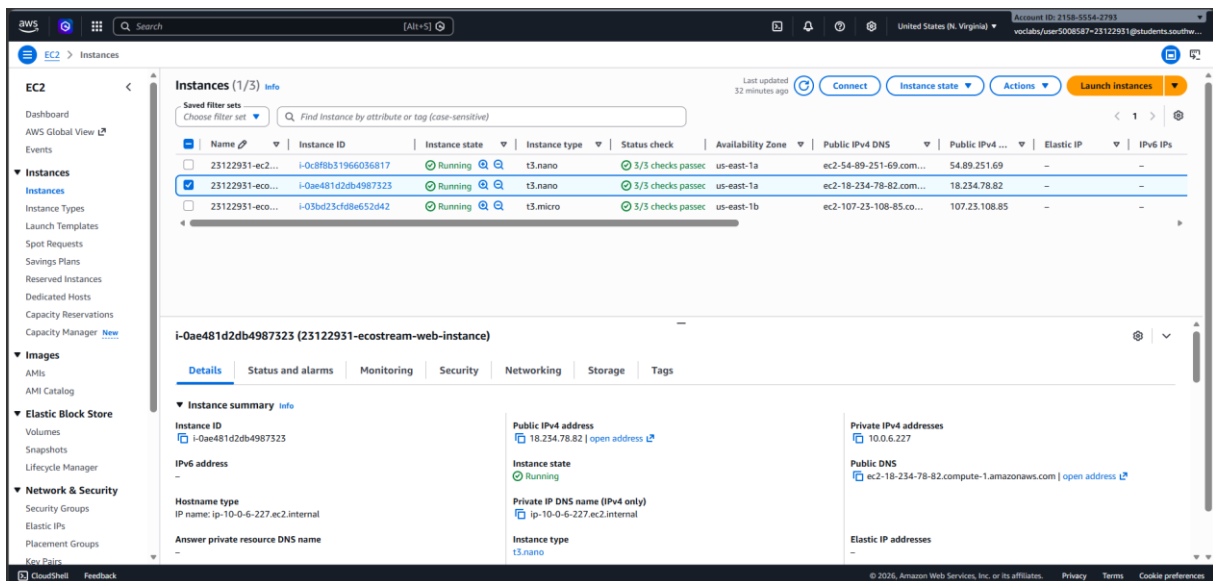
i-Oae481d2db4987323 (23122931-ecostream-web-instance)
PublicIPs: 18.234.78.82 PrivateIPs: 10.0.6.227

Step 14: The EC2 Instances page was accessed to verify that the newly created EcoStream web server instance had been successfully launched and was running. The instance named “23122931-ecostream-web-instance” was selected from the Instances dashboard with its

status showing “Running” with all status checks passed, confirming that the virtual server was functioning correctly within the created VPC environment. The public IPv4 address was then copied and opened in a web browser to test external connectivity to the deployed web application.

The browser successfully displayed the custom EcoStream welcome message hosted on the Apache web server, confirming that:

- the EC2 instance was publicly accessible,
- the Apache HTTP server was running correctly,
- HTTP traffic was permitted through the configured security group,
- and the web application deployment was successful.



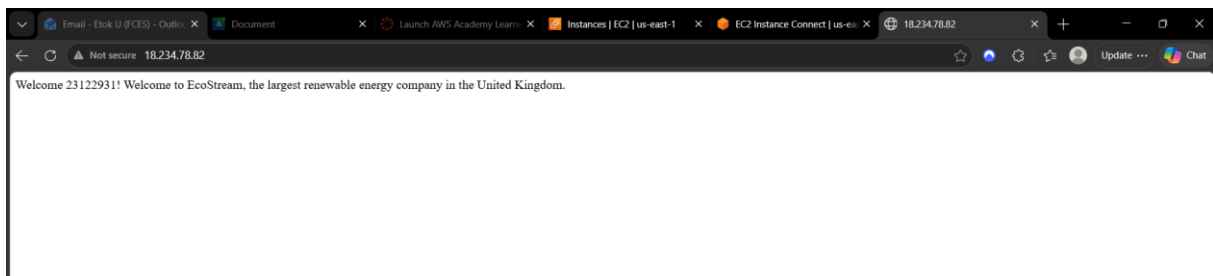
The screenshot shows the AWS Management Console for the 'Instances' page. A table lists three EC2 instances, all in a 'Running' state with '3/3 checks passed'. The instance 'i-0ae481d2db4987323' is highlighted. Below the table, the details for this instance are shown, including its public IPv4 address (18.234.78.82) and public DNS name (ec2-18-234-78-82.compute-1.amazonaws.com).

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
23122931-ec2...	i-0c8f9b31966036817	Running	t3.nano	3/3 checks passed	us-east-1a	ec2-54-89-251-69.com...	54.89.251.69	-	-
23122931-eco...	i-0ae481d2db4987323	Running	t3.nano	3/3 checks passed	us-east-1a	ec2-18-234-78-82.com...	18.234.78.82	-	-
23122931-eco...	i-03bd23c4d8e652d42	Running	t3.micro	3/3 checks passed	us-east-1b	ec2-107-23-108-85.co...	107.23.108.85	-	-

i-0ae481d2db4987323 (23122931-ecostream-web-instance)

Instance summary

- Instance ID: i-0ae481d2db4987323
- Public IPv4 address: 18.234.78.82 | [open address](#)
- Private IPv4 addresses: 10.0.6.227
- Instance state: Running
- Public DNS: ec2-18-234-78-82.compute-1.amazonaws.com | [open address](#)
- Private IP DNS name (IPv4 only): ip-10-0-6-227.ec2.internal
- Elastic IP addresses: -
- Hostname type: -
- IP name: ip-10-0-6-227.ec2.internal
- Answer private resource DNS name: -
- Instance type: t3.nano



The screenshot shows a web browser window with the address bar displaying '18.234.78.82'. The page content shows a welcome message: 'Welcome 23122931!! Welcome to EcoStream, the largest renewable energy company in the United Kingdom.'